

Almanac

The complete documentation, as one book

Shravan Goswami <https://shravangoswami.com>

Contents

1	Introduction	1
1.1	What works today	1
1.2	Beyond the web page	1
1.3	What is still missing	1
1.4	Where the pieces live	1
1.5	Next steps	2
2	Installation	3
2.1	Start a new site, once it is published	3
2.2	Add it to an existing Astro project	3
2.3	Run the dev server	4
2.4	Build for production	4
3	Quick start	5
3.1	1. Set the site identity	5
3.2	2. Shape the sidebar	5
3.3	3. Pick a theme	5
3.4	4. Decide whether you want a blog	5
3.5	5. Replace the favicon	5
3.6	6. Write your first real page	5
3.7	Next steps	6
4	Writing docs	7
4.1	Pages live in a top-level directory	7
4.2	Frontmatter	7
4.3	Headings populate the table of contents	8
4.4	Linking between docs pages	8
4.5	Callouts	8
4.6	Sidebars	8
5	Search	9
5.1	How the index is built	9
5.2	Controlling what gets indexed	9
5.3	Trying it locally	9
6	Theming	10
6.1	What ships	10
6.2	Configuring it	10
6.3	The token system	10
6.4	The picker	10
6.5	The theme-color meta tag	11
6.6	Overriding the tokens	11
7	Terminal demos	12
7.1	<PromptCast />	12
7.2	Removing it	12
8	Executable code	13
8.1	Errors do not break the build	13
8.2	Options	13
8.3	Caching	14
8.4	Python	14
8.5	R	14
8.6	Jupyter kernels	15
8.6.1	Rich output	15

8.7	What is not here yet	15
9	MyST syntax	16
9.1	Install the toolchain	16
9.2	Cross references	16
9.3	Numbered figures	16
9.4	Directives and roles	16
9.5	Math	17
9.6	Executable blocks keep working	17
9.7	What is different in MyST mode	17
9.8	What is not built yet	17
10	PDF output	18
10.1	Turning it on	18
10.2	What you get	18
10.3	Deliberate limitations	18
10.4	The whole thing as a book	19
10.5	Customizing the output	19
10.6	Speed	20
11	Versions and translations	21
11.1	Versions	21
11.2	Translations	21
11.3	Both at once	22
11.4	The picker	22
11.5	Limitations worth knowing	22
12	Link checking	23
12.1	What gets checked	23
12.2	What gets skipped	23
12.3	Why it runs on the output	23
12.4	Reading the report	23
13	Configuration reference	24
13.1	Site	24
13.2	docs	24
13.3	blog	25
13.4	theme	25
13.5	search	25
13.6	Overrides and flags	25
13.7	Sidebar entries	27
13.8	Not wired up yet	27
14	Deployment	28
14.1	site and base	28
14.2	GitHub Pages	28
14.3	What CI has to get right	29
14.4	Sitemap	29
14.5	Analytics	29

1 Introduction

Almanac is a publishing framework for technical and scientific writing. It ships as an Astro integration: you install it from npm, configure it in `astro.config.mjs`, and write Markdown in a top-level `docs/` directory. One source tree produces a documentation site and a blog, and eventually executed code output and print formats.

This site is built with Almanac, so what you are reading is the framework rendering its own documentation.

1.1 What works today

- **Docs routing:** every Markdown file under `docs/` becomes a page, with no per-file layout wiring and no route to add.
- **Sidebars:** group pages explicitly in config, or leave sidebar out and Almanac groups them by directory. See [Writing docs](#).
- **A previous/next pager** at the foot of each docs page, ordered by the sidebar.
- **A table of contents** built from each page's `##` and `###` headings, with scroll-spy highlighting of the section you're reading.
- **Instant search** powered by [Pagefind](#), on a header button and `Cmd+K`. No external service, no API key.
- **A blog** with tags, an RSS feed, hero images, and reading times. Off by default, turned on with `blog.enabled`.
- **42 built-in themes**, a light/dark toggle, and a theme picker. See [Theming](#).
- **A build-time OG image** rendered with [Satori](#), plus JSON-LD structured data and a generated `robots.txt`.
- **Copy-to-clipboard code blocks:** every fenced block gets a copy button.
- **An optional terminal-recording player**, `<PromptCast />`, for documenting CLIs.
- **Config validated with Zod**, so a typo fails the build with the path of the problem instead of rendering a broken page.

1.2 Beyond the web page

These are built and off by default, because each one is a bigger commitment than rendering Markdown:

- **Executable code blocks.** Run JavaScript, TypeScript, Python, and R during the build and embed the real output, cached on the code, the language, and the runtime version. Behind `future.execute`.
- **MyST parsing.** Cross references that write their own text, numbered figures, directives, and roles. Behind `future.myst`.
- **PDF output.** Every docs page typeset by Typst from the same tree the web page came from, executed output included, plus the whole thing as one book with a cover and a clickable table of contents. Styling is yours, down to a full Typst template. Behind `future.pdf`.
- **Versions and translations.** Both inherit unchanged pages instead of copying them, and build in a single pass.

1.3 What is still missing

- **Installing packages into a runtime.** Pyodide gets the Python standard library and WebR gets base R. Anything beyond that needs a [Jupyter kernel](#) with the packages already installed.
- **Rich output from the local runtimes.** A Jupyter kernel's images, SVG, HTML, and JSON are rendered; Pyodide and WebR emit text only.
- **Citations and bibliographies.** `{cite}` parses but there is no bibliography renderer.

1.4 Where the pieces live

Almanac is a package you depend on, not a repository you clone. Two things belong to you:

`astro.config.mjs`, which holds the config, and the content directories (`docs/`, plus `blog/` if you enable it). Everything else, the routes, layouts, components, themes, and search wiring, comes from the package and updates when you update the dependency.

1.5 Next steps

- [Installation](#) covers both a new site and adding Almanac to an existing Astro project.
- [Quick start](#) is the short list of things to change to make the site yours.
- [Configuration reference](#) documents every option, its default, and what is not wired up yet.

2 Installation

Almanac needs Node 22.12 or newer and Astro 7.

Almanac is not on npm yet, so the commands on this page do not work today. Until the first release, clone the repository and start from the bundled starter, as shown below. This site is built with that same code.

== Start from the repository, today

```
git clone https://github.com/shravanngoswamii/almanac.git
cd almanac
pnpm install
pnpm --filter starter dev
```

starter/ is a complete minimal site: an `astro.config.mjs`, a `src/content.config.ts`, a homepage, and a docs/ directory with two example pages. Copy that directory somewhere else and point its almanac dependency at your checkout, and you have your own project.

2.1 Start a new site, once it is published

```
npm create almanac@latest my-docs
cd my-docs
npm run dev
```

The scaffolder writes the same tree that starter/ holds. That is the whole starting point.

2.2 Add it to an existing Astro project

Once it is published, install the framework and its peer dependencies. Satori and resvg render the OG image, Pagefind builds the search index:

```
npm install almanac satori @resvg/resvg-js pagefind
```

Register the integration:

```
// astro.config.mjs
import { defineConfig } from "astro/config";
import almanac from "almanac";

export default defineConfig({
  site: "https://example.com",
  integrations: [
    almanac({
      title: "My Project",
      social: { github: "https://github.com/you/your-project" },
    }),
  ],
});
```

Declare the collections Almanac renders. The loaders point at top-level directories, so writers never touch src/:

```
// src/content.config.ts
import { defineCollection } from "astro:content";
import { blogLoader, blogSchema, docsLoader, docsSchema } from "almanac/content";

export const collections = {
  docs: defineCollection({ loader: docsLoader(), schema: docsSchema() }),
  blog: defineCollection({ loader: blogLoader(), schema: blogSchema() }),
};
```

Then write docs/index.md with a title in its frontmatter, and the route exists.

2.3 Run the dev server

`npm run dev`

This serves the site at `http://localhost:4321`, under whichever base your `astro.config.mjs` sets (see [Deployment](#)).

2.4 Build for production

`npm run build`

Astro compiles the site to static HTML, CSS, and JS in `dist/`, and then Almanac's `astro:build:done` hook runs Pagefind over that output to write the search index into `dist/pagefind/`. You don't chain the two commands yourself.

If Pagefind can't run, the build logs a warning and finishes anyway. You get a working site without search rather than a failed deploy.

== Preview the production build

`npm run preview`

This serves `dist/`, including the search index, so you can check the real output before deploying.

Search only works against a production build. The dev server has no index, and the search dialog deliberately skips loading Pagefind in dev, so the box will open and return nothing. Use ``npm run build && npm run preview`` to test search locally.

3 Quick start

Once Almanac is [installed](#) and the dev server runs, here's the order to work through to turn the starting point into your own site. Almost all of it happens in `astro.config.mjs`.

3.1 1. Set the site identity

The integration's options drive the header, the meta tags, the footer, and the OG image:

```
almanac({
  title: "My Project",
  tagline: "Short line under the title.",
  description: "Used for meta tags and the OG image.",
  author: { name: "You", url: "https://example.com" },
  social: {
    github: "https://github.com/you/your-project",
    sponsor: "https://github.com/sponsors/you",
  },
})
```

Every field is listed in the [Configuration reference](#).

3.2 2. Shape the sidebar

Either describe the navigation explicitly:

```
docs: {
  sidebar: [
    { label: "Get started", items: ["index", "start/installation"] },
    { autogenerate: { directory: "reference" } },
  ],
}
```

Or delete sidebar entirely and let Almanac group pages by their top-level directory. See [Writing docs](#) for how both modes resolve.

3.3 3. Pick a theme

`theme.default` is the theme a first-time visitor sees, and `theme.include` narrows what the picker offers:

```
theme: { default: "almanac-light", include: ["dracula", "nord"] }
```

All 42 built-in ids are listed in [Theming](#).

3.4 4. Decide whether you want a blog

The blog is off by default, because a docs site without one is the common case:

```
blog: { enabled: true, tags: true, rss: true }
```

With it on, posts live in a top-level `blog/` directory and the blog index, tag pages, and `/rss.xml` are all injected for you.

3.5 5. Replace the favicon

The layout serves `/favicon.svg`, so drop your own file at `public/favicon.svg`. The header and footer logo is a built-in mark, not a file you supply.

3.6 6. Write your first real page

Create a `.md` file under `docs/`, give it a title in its frontmatter, and the route exists. If you configured an explicit sidebar, add the page's id to it too, otherwise the page is reachable by URL but invisible in the navigation.

3.7 Next steps

- [Writing docs](#) covers routing, frontmatter, callouts, and sidebars.
- [Configuration reference](#) documents every option and its default.
- [Deployment](#) covers site, base, and static hosting.

4 Writing docs

4.1 Pages live in a top-level directory

Docs content sits in `docs/` at the project root, not under `src/`. `docsLoader()` globs it into the docs collection, and Almanac injects one catch-all route that renders every entry. Adding a page means adding a file:

- `docs/index.md` → `/docs/`
- `docs/guides/writing-docs.md` → `/docs/guides/writing-docs/`
- `docs/reference/configuration.md` → `/docs/reference/configuration/`

An index file serves its directory. `docs.path` moves the directory, and `docs.base` moves the URL prefix (set `base: ""` to serve docs at the site root).

Files and directories starting with an underscore are skipped, so `docs/_partials/note.md` is not a page. `.md`, `.mdx`, and `.mdoc` are all globbed, but MDX and Markdoc only work if you also add the matching Astro integration.

4.2 Frontmatter

`title` is the only required field:

```
---
title: Writing docs
description: Shown under the title and used for meta tags and search.
slug: custom/url           # overrides the URL derived from the file path
sidebar:
  label: Writing           # shorter label for the sidebar
  order: 2                 # lower sorts first
  badge: New               # or { text: "New", variant: "tip" }
  hidden: false            # keep the page but drop it from the sidebar
draft: false               # true keeps the page out of the build
---
```

`title` and `description` show up in the page header, the browser tab title, the JSON-LD breadcrumb data, and the Pagefind index, so keep both short and accurate.

The schema also accepts these, and all of them are honoured:

Field	Effect
<code>template</code>	"splash" drops the sidebar, table of contents, pager, and the automatic title header, so the page composes its own hero
<code>tableOfContents</code>	<code>false</code> hides it; <code>{ minDepth, maxDepth }</code> overrides the site-wide depths for this page
<code>prev / next</code>	<code>false</code> removes that pager link; <code>{ link, label }</code> points it somewhere of your choosing
<code>editUrl</code>	<code>false</code> opts the page out of the site-wide edit link; a string replaces it
<code>lastUpdated</code>	A date overrides the git commit date; <code>false</code> hides the line
<code>head</code>	Extra tags in this page's <code><head></code> , merged after the site-wide ones

For example, a landing page with no chrome and a pager that only goes forward:

```
---
title: Welcome
template: splash
prev: false
next: { link: /docs/start/installation/, label: Install it }
---
```

4.3 Headings populate the table of contents

The table of contents on the right of every docs page is generated from the page's `##` and `###` headings, in document order. You never maintain it by hand. As you scroll, an `IntersectionObserver` highlights the heading you're in, and on narrow screens the same list collapses into an "On this page" panel.

Deeper headings (`####` and below) are rendered in the page but left out of the table of contents.

4.4 Linking between docs pages

Link with a path relative to the current page, not a root-relative one:

See `[Theming](../guides/theming/)` for the color system.

Almanac sites are often served under a base path (this one is served under `/almanac`, see [Deployment](#)). A relative link resolves correctly under any base; a root-relative link like `/docs/guides/theming/` skips the base path and 404s in production.

4.5 Callouts

Wrap a paragraph in a `<div>` with class `callout note` or `callout warning` to get a styled admonition. The styles ship with the framework, so there is nothing to import:

```
<div class="callout note">
Notes are for helpful context that isn't essential to following the instructions.
</div>
```

```
<div class="callout warning">
Warnings are for things that break a build or lose data if skipped.
</div>
```

4.6 Sidebars

Leave `docs.sidebar` out of your config and Almanac autogenerates the navigation: pages are grouped by their top-level directory, root-level pages lead under an "Overview" heading, and each group is sorted by `sidebar.order` first and alphabetically by label after. New files appear on their own.

Set `docs.sidebar` and you control the order instead. An entry can be a doc id, an explicit doc with a new label, an external link, a nested category, or an autogenerated directory:

```
sidebar: [
  "index",
  { doc: "start/installation", label: "Install" },
  { link: "https://astro.build", label: "Astro" },
  { label: "Guides", items: ["guides/writing-docs", { label: "Deep", items: ["guides/search"] }] },
  { autogenerated: { directory: "reference", collapsed: true } },
]
```

Ids that don't resolve to a page are dropped rather than failing the build, which means a typo shows up as a missing sidebar link. With an explicit sidebar, adding a page does not add it to the navigation: add its id here too.

The previous/next pager at the foot of each page walks the flattened sidebar, so sidebar order is also reading order. External links are excluded from it.

5 Search

Almanac ships instant search powered by [Pagefind](#). It runs entirely against your built HTML, so there is no external search service, no API key, and no per-query cost.

5.1 How the index is built

The integration hooks `astro:build:done`. Once Astro has written `dist/`, Almanac shells out to Pagefind over that directory:

```
npx -y pagefind --site dist
```

You don't chain that yourself, and you don't add it to your build script. `npm run build` is enough. The result lands in `dist/pagefind/`, which the search dialog loads on first use.

Because Pagefind crawls rendered HTML rather than your source Markdown, there is no index while the dev server is running.

Turn it off with:

```
search: { provider: "none" }
```

If Pagefind fails (no network to fetch it, or a sandboxed CI step), the build logs a warning and still succeeds. You get a site whose search box returns nothing rather than a broken deploy, so check the build log if search goes quiet.

== Opening search

The dialog opens three ways:

- Clicking the search button in the site header
- Pressing `Cmd+K` (macOS) or `Ctrl+K` (Windows and Linux) anywhere on the site
- Tapping the search icon in the mobile menu

Escape closes it, as does clicking the backdrop. Pagefind's UI script and stylesheet are only fetched the first time you open the dialog, so search costs nothing on pages nobody searches from.

5.2 Controlling what gets indexed

Two data attributes decide what Pagefind sees, and Almanac's layouts already set both:

- `data-pagefind-body` marks indexable content. It's on the main content region and on the docs article, so page bodies are searchable out of the box.
- `data-pagefind-ignore` excludes an element. It's on the header, footer, sidebar, table of contents, mobile panels, and the search UI itself, so navigation labels and repeated chrome don't crowd out real results.

You only need to touch either one if you add your own chrome that shouldn't be searchable, or a new content region that should be.

5.3 Trying it locally

Running the dev server and typing in the search box returns nothing. There is no index, and the dialog skips loading Pagefind in dev on purpose. Build first.

```
npm run build && npm run preview
```

That builds the site including the index and serves it the way production will, so you can confirm search works before you deploy.

6 Theming

6.1 What ships

Almanac has 42 built-in themes, every one of them available in both light and dark:

- The **default family**, almanac-light and almanac-dark.
- **Twelve color families**: slate, graphite, blue, indigo, violet, sky, teal, emerald, amber, rose, crimson, fuchsia.
- **Eight signature palettes**: sepia, solarized, gruvbox, dracula, nord, onedark, tokyonight, catppuccin.

Ids are <family>-<mode>, so dracula-dark, teal-light, and almanac-dark are all valid. Every family exists in both modes, which the test suite enforces, so the light/dark toggle never has to drop a visitor into a different palette.

6.2 Configuring it

```
theme: {  
  default: "almanac-light",  
  include: "all",  
}
```

default is what a visitor sees before they pick anything. include accepts "all" or an array; array entries can be full ids or bare family names, so ["dracula", "nord-dark"] gives you both Dracula modes plus dark Nord. The almanac family is always kept regardless, so there is always something to fall back to.

6.3 The token system

A theme is a set of CSS custom properties. The default family's tokens live in the framework's stylesheet under :root and :root[data-mode="dark"]; every other theme is derived from a four-color seed (background, surface, text, accent) and emitted as an override block:

```
html[data-mode="dark"][data-theme="dracula-dark"] {  
  --bg: #282a36;  
  --surface: #343746;  
  --surface-muted: #484a57;  
  --text: #f8f8f2;  
  --muted: #a5a6a7;  
  --line: #575a65;  
  --accent: #bd93f9;  
  --accent-strong: #e1cef3;  
  /* ... */  
}
```

Deriving the rest of the token set from four colors is why adding a theme is a handful of colors instead of a dozen hand-tuned properties. Every component reads these variables, so overriding --accent in one place retints the whole site.

Only the themes you included are emitted, and they are inlined in a single <style> in the head, so switching themes never fetches anything.

6.4 The picker

The header has two controls: a quick toggle and a caret that opens the picker.

- The **quick toggle** flips to the same family's opposite mode, so nord-light becomes nord-dark rather than jumping to the default palette.
- The **picker** has a System / Light / Dark segment and a grid of preview tiles, one per included theme in the current mode.

A choice is written to `localStorage` under the key `theme`, holding either a theme id or the literal `system`. Under `system`, the site follows `prefers-color-scheme` and reacts live when the OS setting changes.

To avoid a flash of the wrong palette, an inline script in the head runs before first paint: it reads the stored value, resolves `system` against the media query, and sets `data-mode` and `data-theme` on `<html>` before the rest of the page renders.

6.5 The `theme-color` meta tag

You don't configure `<meta name="theme-color">`. Almanac derives it from your default theme: the light value comes from that family's accent, the dark value from its background. Change `theme.default` and the color a mobile browser tints its own chrome with follows automatically.

6.6 Overriding the tokens

The tokens are plain custom properties, so your own stylesheet can redefine any of them. Match the framework's specificity when you target a specific theme:

```
html[data-mode="light"][data-theme="almanac-light"] {  
  --accent: #7c3aed;  
}
```

``theme.customCss`` is accepted by the config schema, but the framework does not layer those files in yet. Until it does, add your stylesheet the normal Astro way, by importing it from a page or component you own.

7 Terminal demos

If your project is a CLI or another dev tool, the clearest documentation is often a real terminal session rather than a wall of instructions. Almanac ships an optional component for exactly that. Nothing else in the framework depends on it, and it costs you nothing if you never import it.

7.1 <PromptCast />

PromptCast plays back an [asciinema](#) recording (a `.cast` file) with the `asciinema-player` package, so readers see the session replay rather than a static screenshot.

It is not bundled with the framework's dependencies, so install the player yourself:

```
npm install asciinema-player
```

Then import the component and drop it in:

```
---
import PromptCast from "almanac/components/PromptCast.astro";
---

<PromptCast />
```

The component takes no props. It plays `public/casts/quickstart.cast`, resolved against your site's base path, and autoplays with a loop that restarts two seconds after the recording ends. The player is themed from the same CSS custom properties as the rest of the site, so it follows whichever theme the visitor picked.

Plain `.md` files can't import components. Use `<PromptCast>` from an `.astro` page, or from an `.mdx` page if you have added Astro's MDX integration.

== Recording your own

1. Install the [asciinema CLI](#) if you don't already have it.
2. Record a session:

```
asciinema rec quickstart.cast
```

Run through the workflow you want to show, then press `Ctrl+D` or type `exit` to stop recording.

1. Move the file to `public/casts/quickstart.cast`. That path is fixed in the component, so keeping the name means there is nothing else to change.
2. Keep it short. A cast that runs longer than about a minute stops being a demo and starts being a video nobody watches.

7.2 Removing it

If your project has no terminal in it, delete `public/casts/`, remove your `<PromptCast />` usage, and uninstall `asciinema-player`. Search, navigation, theming, and everything else described in these docs are unaffected.

8 Executable code

Almanac can run a fenced code block during the build and put its output on the page. This is opt-in, because executing code is a bigger commitment than rendering it.

Turn it on in `astro.config.mjs`:

```
almanac({
  title: "My Project",
  future: { execute: true },
})
```

Then mark a block with `exec`:

```
```js exec
const rows = [1, 2, 3].map((n) => n ** 3);
console.log(rows.join(", "));
```
```

Which renders the source followed by its result:

```
const rows = [1, 2, 3].map((n) => n ** 3);
console.log(rows.join(", "));
```

1, 8, 27

TypeScript works too, with types stripped before running:

```
interface Point {
  x: number;
  y: number;
}

const distance = (a: Point, b: Point): number =>
  Math.hypot(b.x - a.x, b.y - a.y);

console.log(distance({ x: 0, y: 0 }, { x: 3, y: 4 }));

5
```

8.1 Errors do not break the build

A block that throws reports the failure in place and the build carries on, so one broken example cannot stop you shipping:

```
JSON.parse("{ not valid json }");
```

```
<anonymous_script>:1
{ not valid json }
^
```

```
SyntaxError: Expected property name or '}' in JSON at position 2 (line 1 column 3)
at JSON.parse (<anonymous>)
at file:///almanac/block.mjs:1:6
at ModuleJob.run (node:internal/modules/esm/module_job:343:25)
at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:681:26)
at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:117:5)
Node.js v22.23.1
exited with code 1
```

8.2 Options

Add these to the fence after `exec`:

| Option | What it does |
|--------|--------------|
|--------|--------------|

| Option | What it does |
|--------------|---|
| hide-code | Show only the output |
| hide-output | Run the block but render nothing |
| timeout=5000 | Override the 30 second default, in milliseconds |

A block that only sets something up can hide its output:

```
globalThis.__unused = "this block runs but shows nothing";
```

8.3 Caching

Results are cached in `.almanac/exec/`, keyed by a hash of the code, the language, the runtime version, and any declared dependencies. An unchanged block is never run twice, and a Node upgrade invalidates every entry, because output that depends on the runtime should not survive it.

Commit the cache directory to make CI builds reproducible and fast, or leave it out of version control and let CI cache it.

8.4 Python

Python runs too, through [Pyodide](#). It is an optional peer dependency, so install it only in projects that want it:

```
npm install pyodide
```

Then a python block behaves like any other:

```
```python exec
import statistics

samples = [12, 15, 9, 22, 17]
print("mean: ", statistics.mean(samples))
print("stdev: ", round(statistics.stdev(samples), 3))
```

import statistics

samples = [12, 15, 9, 22, 17]
print("mean: ", statistics.mean(samples))
print("stdev: ", round(statistics.stdev(samples), 3))

mean: 15
stdev: 4.95
```

The whole Python standard library is available. Third party packages are not installed for you yet, so numpy and friends need Pyodide's own package loading, which Almanac does not drive.

Pyodide starts in about a second, and it starts once per block that actually runs. Cached blocks cost nothing, so this shows up on the first build and then mostly disappears.

Python runs in its own process for the same reason JavaScript does, and it matters more here: a WASM runtime cannot be interrupted from the thread it runs on, so an infinite loop would hang the build rather than time out.

8.5 R

R runs through [WebR](#), also an optional peer:

```
npm install webr
```

```
samples <- c(12, 15, 9, 22, 17)
cat("mean: ", mean(samples), "\n")
cat("sd:   ", round(sd(samples), 3), "\n")
print(summary(samples))
```

```
mean: 15
sd: 4.95
Min. 1st Qu. Median Mean 3rd Qu. Max.
9 12 15 15 17 22
```

Autoprint works the way it does in an R console, so a bare expression prints its value. Warnings appear as warnings and do not fail the block; a `stop()` is reported as an error.

WebR starts faster than Pyodide, around half a second. Installing R packages from a repository is not wired up, so the base R distribution is what you get.

8.6 Jupyter kernels

Pyodide and WebR are convenient because they need no setup, and limited for the same reason: you get the base distribution and nothing you installed yourself. When you need a real environment, name a kernel:

```
```python exec kernel=python3
import sys
print("platform:", sys.platform)
```
```

Naming a kernel is what routes the block to a Jupyter server, so a plain `python exec` keeps running locally under Pyodide. The two can sit on the same page.

Point Almanac at the server through the environment:

```
export ALMANAC_JUPYTER_URL=http://127.0.0.1:8888
export ALMANAC_JUPYTER_TOKEN=your-token
```

The token is read from the environment rather than the config file on purpose: a credential in `astro.config.mjs` is a credential in version control.

No extra dependency is needed. Almanac talks to the server's REST and WebSocket API directly, creating a session per block and deleting it afterwards so a build cannot leave kernels running.

If the server is not configured or not reachable, the block reports that where its output would go and the rest of the site builds. That failure is never cached, so fixing the server and rebuilding is enough.

8.6.1 Rich output

A kernel can return more than text, and Almanac keeps it:

| Kernel output | Rendered as |
|------------------|------------------------------------|
| image/png | an inline <code></code> |
| image/svg+xml | inline SVG |
| text/html | the markup, as given |
| application/json | a formatted block |

So a matplotlib figure or a pandas table arrives on the page as a figure or a table. The plain text alternative is kept alongside it, so the output still reads sensibly where an image cannot be shown.

Local runtimes emit text only today. The artifact plumbing is shared, so a Pyodide figure is a matter of teaching that driver to capture one.

8.7 What is not here yet

Installing packages into a runtime. Pyodide gets the Python standard library and WebR gets base R; anything beyond that needs a Jupyter kernel with the packages already installed.

Failures that are the environment's fault rather than the code's, a runtime that is not installed, a driver that dies, a timeout, are never cached. A Python traceback is cached, because the code really does raise it.

9 MyST syntax

Almanac can parse your Markdown into a **MyST** document tree instead of straight to HTML. That buys you the things technical writing needs and Markdown has no syntax for: references that write their own text, figures that number themselves, and directives.

It is off by default. Turn it on with a future flag:

```
// astro.config.mjs
almanac({
  title: "My Project",
  future: { myst: true },
})
```

MyST is a superset of Markdown, so nothing you already wrote stops working.

9.1 Install the toolchain

The MyST packages are optional peer dependencies, so a project that does not use them does not pay for them:

```
npm install myst-parser myst-to-html myst-transforms unified vfile rehype-stringify
```

Turning the flag on without them fails the build with a message naming the package that is missing, rather than silently rendering something different.

9.2 Cross references

Label anything with (name)= on the line above it, then link to it with an empty link text. The text is filled in from whatever you referenced:

```
(setup)=
## Setting things up
```

Later on, see `[](#setup)` and the reader gets "Setting things up".

Reference a figure and you get its number instead. Write your own text and it wins, so `[the setup guide] (#setup)` still says what you wrote.

A reference to a label that does not exist warns during the build and renders as visible text rather than an invisible empty link, so a broken reference is obvious on the page instead of silently disappearing.

9.3 Numbered figures

```
:::{figure} ./chart.png
:label: fig-growth
:alt: Growth over time
```

```
Requests per second, measured weekly.
:::
```

The caption is prefixed with "Figure 1", numbered in document order, and `[](#fig-growth)` renders as a link reading "Figure 1". Relative image paths go through Astro's asset pipeline exactly as they do in plain Markdown mode.

9.4 Directives and roles

Admonitions become directives rather than raw HTML:

```
:::{note}
Directive bodies are full Markdown, so bold and [links](#setup) work.
:::
```

note, tip, warning, caution, important, and danger are all available. Inline roles cover smaller things, such as `{abbr} MyST (Markedly Structured Text)` for an abbreviation with a tooltip.

9.5 Math

Inline $E = mc^2$ and display blocks:

```
$$
\int_0^1 x^2 \, dx = \frac{1}{3}
$$
```

Almanac emits `math-inline` and `math-display` elements holding the source. It does not ship a math renderer, so add KaTeX or MathJax yourself through head if you want typeset output.

9.6 Executable blocks keep working

Both syntaxes run when `future.execute` is also on. MyST's own is `{code-cell}`:

```
```{code-cell} js
console.log("runs at build time");
```
```

And the ordinary fence with `exec` behaves identically, so content moves between the two modes without being rewritten:

```
```js exec
console.log("also runs at build time");
```
```

That works because the fence directives are recovered from the source line the block starts on. MyST's parser keeps a block's language but discards the rest of the info string, so `exec`, `hide-code`, and `timeout=` would otherwise be lost.

9.7 What is different in MyST mode

Worth knowing before you flip the flag:

- **Remote images are not optimized.** Local paths still are. Deciding whether a remote URL is allowed needs one of Astro's internal helpers, which a strict package manager will not expose to a dependency, and claiming an image Astro then refuses would fail the build. Remote images render as ordinary `<img>` tags.
- **Syntax highlighting is unchanged.** Code blocks go through the same Shiki step and honour your `shikiConfig`.
- **Raw HTML still passes through**, so existing `<div class="callout note">` markup keeps rendering.
- **smartyants and gfm toggles do not apply.** Those are options on Astro's remark pipeline. MyST does its own typography and always supports tables, strikethrough, and task lists.

9.8 What is not built yet

Citations parse but there is no bibliography renderer: `{cite}` needs a `.bib` file and a citation style, and neither is wired up. Cross-document references, the ones that point into another page, resolve within a page only. Both need the cross-collection content phase that docs versioning also needs.

10 PDF output

Almanac can typeset every docs page as a PDF during the build. Not a print stylesheet or a headless browser screenshot: a real typeset document, produced by [Typst](#) from the same tree the web page was rendered from.

This site has it on. Every documentation page has a PDF link at the bottom, and the sidebar has one for the whole thing as a book.

Executable code

is the interesting page: its PDF holds the same computed Python and R output the page shows.

That last part is the point. The PDF and the page cannot disagree, because there is one parse, one set of resolved cross references, and one set of executed results feeding both.

10.1 Turning it on

```
almanac({
  title: "My Project",
  future: { myst: true, pdf: true },
})
```

pdf requires myst, and the build says so if you forget:

```
[almanac] Invalid configuration:
- future.pdf: future.pdf needs future.myst: the PDF is built from the MyST tree
```

Install the toolchain, which is an optional peer like the rest:

```
npm install myst-to-typst @myriaddressmin/typst-ts-node-compiler
```

No system Typst installation is needed. The compiler is a native module with Typst built in.

10.2 What you get

One PDF per docs page, written to `dist/pdf/<page id>.pdf`, where the id is the same one the sidebar uses. So `docs/guides/writing.md` becomes `/pdf/guides/writing.pdf`.

Each page in the sidebar gains a PDF link next to “Edit this page”. The link appears only when the feature is on.

What survives into the document:

- **Headings, lists, tables, and quotes**, typeset rather than screenshotted.
- **Math**, properly set. Inline and display both.
- **Numbered figures with captions**, using the same numbers the page shows.
- **Cross references**, with the text they resolved to.
- **Admonitions**, as coloured blocks.
- **Executed output**, printed under the code that produced it.

10.3 Deliberate limitations

- **Remote images are not embedded.** Typst cannot fetch over the network, and a build that reaches out to load a figure is a build that fails in CI behind a firewall. A remote image becomes a visible `[image: url]` line, and the build logs which page it was on. Local images are embedded normally.
- **A page that fails to typeset does not fail the build.** It is reported with the compiler’s own diagnostics and skipped. The HTML for that page was already correct, and losing the whole site because one document confused the typesetter is the wrong trade.
- **Only docs pages.** Blog posts are not typeset.

- **Only the default version and locale.** A variant page can share its source file with the page it inherits from, so a variant PDF would silently be a copy of another one. The link is hidden there rather than pointing at something wrong.

10.4 The whole thing as a book

One PDF holding every page, with a cover, a clickable table of contents, and numbered chapters:

```
almanac({
  pdf: {
    book: {
      enabled: true,
      filename: "handbook.pdf",
      subtitle: "The complete documentation",
      toc: { depth: 3 },
    },
  },
  future: { myst: true, pdf: true },
})
```

What you get in `dist/pdf/handbook.pdf`:

- **A cover page** with the title, subtitle, author, site, and build date.
- **A table of contents** with real page numbers and dot leaders, whose entries are clickable. It is Typst's own outline, so it cannot drift from the content.
- **Numbered headings**, 1, 1.1, 1.1.1, restarting the page count after the front matter so page 1 is the first chapter rather than the cover.
- **PDF bookmarks**, so a reader's outline pane is populated without anything extra.

Chapter order follows your sidebar, because that is the reading order you already chose. Pages the sidebar does not mention still appear, sorted, after the ones it does; nothing is silently dropped. Override it explicitly if you want:

```
book: { enabled: true, order: ["index", "start/installation", "guides/writing"] }
```

Set `perPage: false` alongside it to publish only the book.

10.5 Customizing the output

Two levels. The structured options cover the common cases:

```
pdf: {
  paper: "us-letter",
  margin: "(x: 2cm, y: 2.5cm)",
  bodyFont: ["Georgia", "Liberation Serif"],
  monoFont: ["JetBrains Mono"],
  bodySize: "11pt",
  accent: "#b45309",
  numberHeadings: true,
}
```

`margin` is passed to Typst verbatim, so anything Typst accepts works there.

Beyond that, replace the template entirely. A template is a Typst file, and whatever it does is what the document does:

```
pdf: {
  template: "./pdf/page.typ",
  book: { enabled: true, template: "./pdf/book.typ" },
}
```

Almanac defines a Typst dictionary named `almanac` before your template runs, so the template has the metadata it needs:

| Field | Contents |
|-------------------------------|--|
| <code>almanac.title</code> | Page title, or the book title |
| <code>almanac.subtitle</code> | Page description, or the book subtitle |
| <code>almanac.author</code> | From <code>author.name</code> |
| <code>almanac.source</code> | Your site URL |
| <code>almanac.kind</code> | "page" or "book" |
| <code>almanac.date</code> | Build date, YYYY-MM-DD |
| <code>almanac.chapters</code> | Chapter titles in order, for a book |

A complete per-page template:

```
// pdf/page.typ
#set page(paper: "a5", margin: 1.4cm, numbering: "1")
#set text(size: 9pt, font: ("DejaVu Sans",))
#set par(justify: true)
#show link: set text(fill: rgb("#b45309"))

#if almanac.title != "" [
  #block(fill: rgb("#f5f2e8"), inset: 10pt, width: 100%, radius: 3pt)[
    #text(size: 15pt, weight: "bold")[#almanac.title]
    #if almanac.subtitle != "" [
      \ #text(size: 8pt, fill: luma(90))[#almanac.subtitle]
    ]
  ]
  #v(0.8em)
]
```

A custom template replaces the styling and the title block, and nothing else. The handful of definitions `myst-to-typst` requires are always emitted, so a template can be three lines without failing to compile.

An empty template file is meaningful: it strips the built-in furniture and leaves Typst's defaults.

10.6 Speed

The Typst compiler is created once per build and reused across pages, because it carries a font book and a parsed standard library that cost real time to build. A site of twenty pages typesets in a couple of seconds after that.

Nothing about the PDF step is cached between builds. Unlike code execution, where skipping unchanged work is the whole feature, typesetting is fast enough that a cache would mostly add ways to be wrong.

11 Versions and translations

Older versions and other languages are the same problem twice: a directory that shadows your docs, served under a URL prefix, whose missing pages fall back to what they shadow. Almanac treats them as one mechanism, which is why neither one requires copying your content.

11.1 Versions

Declare them newest first:

```
almanac({
  title: "My Project",
  versions: [
    { name: "1.0", label: "1.0 (old)" },
  ],
})
```

Then create `versioned_docs/version-1.0/` and put in it **only the pages that version changed**:

```
docs/
  index.md
  guides/writing.md
  guides/theming.md
versioned_docs/
  version-1.0/
    index.md          <- the only page 1.0 changed
```

That gives you:

| URL | Content |
|--|--|
| <code>/docs/</code> | current <code>docs/index.md</code> |
| <code>/docs/1.0/</code> | <code>versioned_docs/version-1.0/index.md</code> |
| <code>/docs/guides/writing/</code> | current |
| <code>/docs/1.0/guides/writing/</code> | inherited from current, not copied |

The inheritance is the point. Docusaurus snapshots every file for every version, which is why [its own documentation opens by suggesting you avoid versioning](#). Here an old version costs a directory holding its differences, and adding a version does not multiply your repository.

The trade is honest: an inherited page shows the current text under an old version's URL. If a page changed, branch it. If it did not, inheriting is correct and free.

11.2 Translations

```
almanac({
  title: "My Project",
  i18n: {
    defaultLocale: "en",
    locales: [
      { code: "en", label: "English" },
      { code: "fr", label: "Français" },
      { code: "ar", label: "العربية", dir: "rtl" },
    ],
  },
})
```

Translations live in `i18n/<code>/docs/`, mirroring your docs tree:


```
docs/index.md
i18n/fr/docs/index.md    <- translated
                           <- everything else falls back to English
```

The default locale keeps its plain URLs. Others are prefixed: `/docs/fr/`. An untranslated page falls back to the default locale rather than 404ing, so a partial translation is a usable site from the first file.

Each page declares its own `lang`, and a locale marked `dir: "rtl"` sets the direction on the html element so the browser lays the page out correctly.

11.3 Both at once

Versions and locales combine. `/docs/fr/1.0/guides/writing/` resolves in order of specificity: the French 1.0 file, then the French current file, then the English 1.0 file, then the English current file. The first one that exists wins.

11.4 The picker

A version and language selector appears in the docs sidebar when either feature is configured. Switching keeps you on the same page when it exists in the target variant, and lands on that variant's home page when it does not, because sending a reader to a 404 is worse than sending them one level up.

Like every other component, it can be replaced:

```
almanac({
  components: { VariantPicker: "./src/components/MyPicker.astro" },
})
```

11.5 Limitations worth knowing

- **PDFs are only generated for the current version in the default locale.** A variant page can share its source file with the page it inherits from, and the Markdown processor is handed a file path with no way to tell which variant it is rendering, so a variant PDF would silently be a copy of another one. The PDF link is hidden on variant pages rather than pointing at something wrong.
- **Search indexes everything.** Pagefind sees every variant, so a search can return the 1.0 page and the current one. Scoping search per variant needs an index per variant, which is not built.
- **Only docs are versioned or translated.** The blog is not.
- **UI strings are not translated.** “On this page”, “Edit this page”, and the rest are English. Only your content is localized.

12 Link checking

Documentation rots through its links first. A page gets renamed, six others keep pointing at the old URL, and nobody notices until a reader does.

Almanac checks every internal link when the build finishes. It is on by default at warn.

12.1 What gets checked

Two things, configured separately:

```
almanac({
  title: "My Project",
  onBrokenLinks: "throw",
  onBrokenAnchors: "warn",
})
```

`onBrokenLinks` covers `<a href>` targets that resolve to no page. `onBrokenAnchors` covers `#fragment` targets that no element on the destination page carries, including fragments on the current page.

Both accept four values:

| Value | Effect |
|----------|--|
| "ignore" | Skip the check entirely. No work is done |
| "log" | Report at info level |
| "warn" | Report as a warning, the default |
| "throw" | Fail the build |

"throw" is the right setting for CI once your links are clean. Getting there first is easier at "warn".

12.2 What gets skipped

External links are not fetched. Anything with a scheme (`https:`, `mailto:`, `tel:`) or a protocol-relative host (`//cdn.example.com`) is left alone. A checker that hit the network would make every build depend on the whole internet's uptime, and it would be slow enough that you would turn it off.

12.3 Why it runs on the output

The check reads the emitted HTML rather than your Markdown. That is deliberate: a link can be broken by a slug override, a base path, or a redirect that only exists after the build, and those are exactly the breakages that reach production. Checking sources would miss all three and would also flag links that are actually fine.

It also means the checker sees links your content did not write, like the ones in the header, sidebar, and pager. When Almanac itself generates a bad link, this catches that too.

Relative hrefs are resolved the way a browser resolves them, against the page's own directory. Trailing slashes are treated as interchangeable, so `/docs/guide` and `/docs/guide/` both find the same page.

12.4 Reading the report

Each entry names the page the link was written on and the href exactly as it appeared:

```
[WARN] [almanac] 2 broken links:
  /docs/start/quick-start/ -> ../../guides/gone/
  /blog/hello/ -> /docs/removed/
```

The href is unresolved on purpose. Seeing `../../guides/gone/` rather than `/guides/gone/` tells you what to search for in your source.

13 Configuration reference

All configuration is the object you pass to `almanac()` in `astro.config.mjs`. It is parsed with Zod, so an unknown shape or a bad type fails the build with the path of the problem (`docs.toc.maxDepth`: Number must be less than or equal to 6) instead of rendering a broken page.

```
// astro.config.mjs
import { defineConfig } from "astro/config";
import almanac from "almanac";

export default defineConfig({
  site: "https://example.com",
  integrations: [almanac({ title: "My Project" })],
});
```

`title` is the only required option.

13.1 Site

| Option | Type | Default | Controls |
|--|--------|-----------------------------|---|
| <code>title</code> | string | required | Header brand, page titles, OG image, RSS feed title |
| <code>tagline</code> | string | none | Footer line and the fallback meta description |
| <code>description</code> | string | none | Meta description and OG image body copy; falls back to tagline, then title |
| <code>author.name</code> | string | none | Footer copyright line |
| <code>author.url</code> | URL | none | Link on the footer author name |
| <code>favicon</code> | string | <code>"/favicon.svg"</code> | Favicon path |
| <code>logo.light</code> / <code>logo.dark</code> / <code>logo.alt</code> | string | none | Image paths that replace the built-in mark. Give both modes and CSS swaps them with no flash |
| <code>social.github</code> | URL | none | Header and footer GitHub links |
| <code>social.sponsor</code> | URL | none | Header and footer “Support” links; omit and they disappear |
| <code>social.x</code> / <code>bluesky</code> / <code>mastodon</code> / <code>discord</code> / <code>email</code> | string | none | Icon links in the footer. email accepts a bare address and gets a <code>mailto:</code> prefix |
| <code>editUrl</code> | string | none | Template for “Edit this page” links, for example <code>"https://github.com/o/r/edit/main/{path}"</code> . Without <code>{path}</code> the source path is appended |
| <code>head</code> | array | <code>[]</code> | Extra tags in every page’s <code><head></code> . Pages can add their own with frontmatter head |

13.2 docs

| Option | Type | Default | Controls |
|----------------------|---------|-------------------|---|
| <code>enabled</code> | boolean | <code>true</code> | Whether the docs route is injected at all |

| Option | Type | Default | Controls |
|-----------------------------|---------|---------|---|
| path | string | "docs" | Directory on disk, relative to the project root |
| base | string | "docs" | URL segment the docs are served under; "" serves them at the site root |
| sidebar | array | omitted | Explicit navigation; leave it out to group by directory |
| pager | boolean | true | Previous/next links |
| toc.minDepth / toc.maxDepth | 1 to 6 | 2 / 3 | Heading depths in the table of contents |
| lastUpdated | boolean | false | Adds a last-updated line, dated from the file's last git commit. Needs full history, so set <code>fetch-depth: 0</code> in CI |

13.3 blog

| Option | Type | Default | Controls |
|--------------|---------|---------|---|
| enabled | boolean | false | Whether the blog routes are injected. Off by default: a docs site without a blog is the common case |
| path | string | "blog" | Directory on disk |
| base | string | "blog" | URL segment |
| postsPerPage | integer | 8 | Posts per index page |
| rss | boolean | true | Injects <code>/rss.xml</code> |
| readingTime | boolean | true | Reading time on cards and posts |
| tags | boolean | true | Injects <code><base>/tags/[tag]</code> pages |

13.4 theme

| Option | Type | Default | Controls |
|-----------|-------------------|-----------------|--|
| default | string | "almanac-light" | Theme used before a visitor picks one |
| include | "all" or string[] | "all" | Which themes ship and appear in the picker. Entries can be full ids or family names; the almanac family is always kept |
| customCss | string[] | [] | Extra stylesheets, linked after the framework's own so they win |

See [Theming](#) for the full list of theme ids.

13.5 search

| Option | Type | Default | Controls |
|----------|----------------------|------------|---|
| provider | "pagefind" or "none" | "pagefind" | Whether the post-build index is generated. See Search |

13.6 Overrides and flags

| Option | Type | Default | Controls |
|--------|------|---------|----------|
|--------|------|---------|----------|

| Option | Type | Default | Controls |
|---|-----------------------------|-----------------------|--|
| components | record | { } | Maps a built-in component name to your own .astro file.
Accepted names: Header, Footer, Sidebar, TableOfContents, Pager, Search, ThemePicker, Logo, BlogCard, PageTitle |
| onBrokenLinks | ignore / log / warn / throw | "warn" | What to do about internal links that resolve to no page. Checked against the built output when the build finishes |
| versions | array | [] | Older documentation versions, newest first.
Each takes name, optional label, optional path.
See Versions and translations |
| i18n.defaultLocale | string | "en" | The locale served without a URL prefix |
| i18n.locales | array | [] | Each takes code, optional label, optional dir, optional path |
| pdf.perPage | boolean | true | Emit one PDF per docs page |
| pdf.template | string | none | Typst template replacing the built-in page styling.
See PDF output |
| pdf.paper / margin / bodyFont / monoFont / bodySize / accent / numberHeadings | various | Typst defaults | Structured styling, for when a full template is more than you need |
| pdf.book.enabled | boolean | false | One PDF of every page, with a cover and a clickable table of contents |
| pdf.book.filename | string | "book.pdf" | Output name inside the pdf directory |
| pdf.book.title / subtitle | string | site title / tagline | Cover text |
| pdf.book.toc.enabled / title / depth | boolean / string / 1-6 | true / "Contents" / 2 | The outline |
| pdf.book.template | string | none | Typst template for the book, cover and outline included |

| Option | Type | Default | Controls |
|------------------------------|-----------------------|---------------|---|
| <code>pdf.book.order</code> | <code>string[]</code> | sidebar order | Chapter order by page id |
| <code>onBrokenAnchors</code> | same | "warn" | Same, for <code>#fragment</code> targets that no element on the destination page carries |
| <code>future.myst</code> | boolean | false | Parse through a MyST AST: cross references, numbered figures, directives, roles. See MyST syntax |
| <code>future.execute</code> | boolean | false | Run fenced blocks marked <code>exec</code> and <code>embed</code> their output. See Executable code |
| <code>future.pdf</code> | boolean | false | Typeset every docs page as a PDF. Requires <code>future.myst</code> . See PDF output |

Unfinished subsystems ship behind `future` flags first, so a minor release can't move the ground under a site that is already published.

13.7 Sidebar entries

An entry is one of five shapes, and categories nest:

```
docs: {
  sidebar: [
    "index", // a doc id
    { doc: "start/installation", label: "Install" }, // relabelled doc
    { link: "https://astro.build", label: "Astro" }, // external link
    { label: "Guides", items: ["guides/search"], collapsed: true }, // category
    { autogenerate: { directory: "reference", collapsed: true } }, // whole directory
  ],
}
```

Any entry may carry a badge string. Ids that don't match a page are dropped rather than failing the build. With sidebar omitted, pages are grouped by their top-level directory, root-level pages first, sorted by frontmatter `sidebar.order` and then alphabetically.

Page-level options live in frontmatter instead; see [Writing docs](#).

13.8 Not wired up yet

Nothing. Every option on this page is honoured by the renderer.

Two features inside `future.myst` are partial, and the [MyST guide](#) says so on the page: citations parse but have no bibliography renderer, and cross references resolve within a page rather than across pages.

14 Deployment

An Almanac site is an ordinary static Astro build. `npm run build` writes HTML, CSS, JS, the OG image, `robots.txt`, and the Pagefind index into `dist/`, and any static host will serve that directory as-is. There is no server component and no deploy step inside the framework.

14.1 site and base

Set both in `astro.config.mjs`, not in the Almanac options:

```
export default defineConfig({
  site: "https://example.com",
  base: process.env.BASE_PATH || "/my-docs",
  integrations: [almanac({ title: "My Project" })],
});
```

`site` is the absolute origin. Canonical links, Open Graph URLs, the RSS feed, and the Sitemap: line in `robots.txt` are all built from it, so a missing or wrong `site` produces links that point at the wrong place.

`base` is the subpath the site is served under, which you need for project-level GitHub Pages. Almanac prefixes every link it generates with the base, so the only thing you have to remember is to keep links inside your Markdown relative (`../guides/search/`, not `/docs/guides/search/`). See [Writing docs](#).

Reading `base` from an environment variable is what makes per-PR previews possible: build the same tree with `BASE_PATH=/my-docs/pr-123` and the preview links resolve inside that subfolder instead of colliding with production.

14.2 GitHub Pages

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      env:
        BASE_PATH: /my-docs
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
```

```
steps:
  - uses: actions/deploy-pages@v4
```

Netlify, Vercel, and Cloudflare Pages need the same two things: `npm run build` as the command and `dist` as the output directory. Drop `base` entirely when the site is served from a domain root.

14.3 What CI has to get right

Two build steps reach outside your repository, and both fail soft, which means a broken deploy looks like a missing feature rather than a red build:

- **Pagefind.** The integration spawns `npx -y pagefind --site dist` after the build. Keep `pagefind` in your `devDependencies` so `npx` resolves the local copy instead of downloading one. If it can't run, the build logs `search index skipped` and finishes, and you ship a site whose search box returns nothing.
- **The OG image.** Satori fetches the Inter font from Google Fonts at build time. Without network access it falls back to a generic sans-serif, so the image still renders, just not in the right typeface.

If either matters to you, `grep` the build log rather than trusting a green check.

14.4 Sitemap

Almanac generates `robots.txt` and points its `Sitemap:` line at `/sitemap-index.xml`, but it does not generate the sitemap itself. Add Astro's official integration alongside it:

```
import sitemap from "@astrojs/sitemap";

integrations: [sitemap(), almanac({ title: "My Project" })],
```

14.5 Analytics

Umami is supported and off unless you opt in. Two build-time environment variables drive it:

- `PUBLIC_UMAMI_WEBSITE_ID`: your website id. Empty means no script tag is rendered at all.
- `PUBLIC_UMAMI_SRC`: the script URL, defaulting to `https://cloud.umami.is/script.js` for self-hosted instances.

Set them as repository variables and pass them into the build step alongside `BASE_PATH`. Nothing is committed to the repo and nothing is added to the page without the id.